

Launch & Showcase

Ship, Present, and Get Hired From Your Project

■ LEARNING OUTCOME

By the end of this module you will have a complete launch strategy for your web project — professional README, demo video, case study, social media announcement, portfolio presentation, and a plan to turn your project into job interviews.

■ Skills You Will Gain

- Write a professional README that sells your project
- Record a compelling demo video and GIF walkthrough
- Write a project case study for your portfolio
- Craft LinkedIn and social media announcements
- Present your project confidently in technical interviews
- Create a project showcase that attracts recruiter attention

■ Building is 70% of the work. Showcasing is the other 30% that determines whether it leads to a job. Every senior developer has a story for every project: the problem, the decisions, the results. Prepare yours.

SECTION 1

Professional README

```
# Project Name -- One Line Value Proposition > [Short description: what it does, who it's for, what makes it special] [![Live Demo](https://img.shields.io/badge/Demo-Live-brightgreen)](https://myapp.vercel.app) [![GitHub](https://img.shields.io/badge/GitHub-Code-blue)](https://github.com/user/repo) [![License: MIT](https://img.shields.io/badge/License-MIT-yellow.svg)](LICENSE) ![App Screenshot or GIF](docs/demo.gif) ## Problem Freelance developers waste hours tracking time across multiple clients with spreadsheets that don't generate invoices automatically. ## Solution TaskFlow is a time-tracking + invoice generation app that automatically calculates project costs and generates professional PDF invoices. ## Features - **Time Tracking**: Start/stop timer with project tagging - **Invoice Generation**: Auto-calculate totals, export professional PDF - **Client Management**: Track multiple clients and projects - **Dashboard**: Revenue charts and project status overview - **Authentication**: Secure login with Google OAuth ## Tech Stack | Category | Technology | |-----|-----| | Frontend | React 18, TypeScript, Vite | | Styling | Tailwind CSS, shadcn/ui | | Backend | Node.js, Express | | Database | PostgreSQL, Prisma ORM | | Auth | JWT, bcrypt, Google OAuth | | Deployment | Vercel (FE), Railway (BE) | | Testing | Vitest, React Testing Library | ## Getting Started ### Prerequisites - Node.js >= 18 - PostgreSQL running locally - Google OAuth credentials (optional) ### Installation ``bash # Clone the repository git clone https://github.com/user/taskflow.git cd taskflow # Install all dependencies npm run install:all # Copy environment files cp backend/.env.example backend/.env cp frontend/.env.example frontend/.env # Fill in your values in both .env files # Then run database migrations cd backend && npx prisma migrate dev # Start development servers npm run dev # starts both frontend and backend `` App runs at: http://localhost:5173 API runs at: http://localhost:3000 ## Environment Variables See ``.env.example`` files for required variables. ## Testing ``bash npm test # run all tests npm run test:coverage # with coverage report `` ## What I Learned - Implementing refresh token rotation for secure sessions - Optimising Prisma queries with compound indexes (reduced API response from 800ms to 45ms) - Setting up Redis caching for frequently accessed data - Building accessible form components with proper ARIA attributes ## License MIT
```

SECTION 2

Demo Video and Case Study

Content Type	What to Include	Tool
Demo GIF	2-3 minute walkthrough showing core features, record at 1080p, convert with ezgif.com	Loom, QuickTime, OBS
Demo Video	3-5 minutes: problem -> features -> code architecture. Post to YouTube (unlisted OK)	Loom, OBS, ScreenFlow
Case Study	Problem, research, tech decisions, challenges overcome, metrics, what you learned	Your portfolio site
LinkedIn Post	Problem you solved + screenshot + 3 key learnings + live link + looking for opportunities	LinkedIn
GitHub Topics	Add relevant topics to repo: react, nodejs, postgresql, fullstack, portfolio-project	GitHub repo settings

SECTION 3

The STAR Interview Framework for Projects

How to Present Any Project in an Interview

SITUATION: What was the context and problem you were solving? **TASK:** What were you specifically responsible for building? **ACTION:** What decisions did you make and why? What was hardest? **RESULT:** What did you deliver? Any metrics? What would you do differently? Prepare this for EVERY project in your portfolio. Interviews are storytelling with technical evidence, not reciting code. The 3 interview questions you **MUST** prepare for every project: 1. Walk me through this project 2. What was the hardest technical challenge? 3. What would you do differently knowing what you know now?

```
# STAR ANSWER EXAMPLE -- TaskFlow Project S: I built a time-tracking app to solve my own problem as a freelancer. I was using a spreadsheet to track hours and manually calculating invoices. The process took 2+ hours per client per month. T: I was responsible for the full stack: React frontend, Node/Express API, PostgreSQL database, authentication, and deployment on Vercel + Railway. A: The hardest challenge was implementing secure authentication. I initially used JWTs stored in localStorage, but realised XSS attacks could steal tokens. I researched the access/refresh token pattern, moved refresh tokens to httpOnly cookies, and implemented token rotation. This took 2 extra days but significantly improved security. For performance, I noticed API responses were slow (~800ms). I used EXPLAIN ANALYZE in PostgreSQL and found missing indexes. Adding a compound index on (user_id, created_at) brought responses to 45ms. R: Delivered a working app with 12+ features in 6 weeks. Lighthouse performance score: 96. All tests passing (87% coverage). The invoice generation feature alone saves me 2 hours/month personally. If I rebuilt it, I would use React Query from day 1 -- managing loading states manually with useEffect was unnecessarily complex. # TECHNICAL QUESTIONS TO PREPARE: - Why did you choose PostgreSQL over MongoDB? - How does your JWT authentication work? - How did you handle CORS between frontend and backend? - What does your CI/CD pipeline do? - What was your biggest bug and how did you debug it? - How would you scale this to 10,000 users?
```

SECTION 4

Launch Checklist and Job Search Strategy

1	Final Code Review Read every file once. Remove console.logs, TODO comments, test data. Consistent formatting.
2	Test on Real Devices Test on iOS Safari, Android Chrome, slow 3G (DevTools throttle). Fix mobile issues.
3	Lighthouse Audit All four scores above 90. Fix every red issue. Screenshot your score for the portfolio.
4	README Polish Every link works, screenshots are high quality, setup steps verified on a clean machine.
5	Demo GIF/Video Record 3-5 minute demo. Show every core feature. Post to YouTube, embed in README.
6	Social Announcement LinkedIn post: problem + solution + link + 3 learnings + hashtags. Tag technologies used.
7	Portfolio Update Add to portfolio with screenshot, tech stack, live URL, and 3-sentence description.
8	GitHub Cleanup Pin repo to profile. Add description, website URL, and topic tags to repository.
9	Apply Strategically Use project as conversation starter. 'I built X which is similar to what you're building.'

Keep Iterating

Add one new feature per week. Project growth shows continuing commitment.

After Launch Action	Why It Matters
Post on LinkedIn with demo GIF	Recruiters and peers see it; often leads to DMs and referrals
Share in Discord/Slack communities	Get feedback, fix bugs, build reputation in developer communities
Write a blog post about a challenge	Shows communication skills; SEO drives additional traffic to portfolio
Add it to job applications	'I built a similar feature' in cover letter + link = instant attention
Submit to Hacker News Show HN	'Show HN: I built X because Y' -- can get hundreds of users and feedback
Tag companies that use your stack	On LinkedIn: 'Built with React and Node.js' -- tags get recruiter attention

■ PRACTICE Q & A

Q1. What should be in every project README?

A: Project name + tagline, live demo URL badge, screenshot/GIF, problem it solves, feature list, tech stack table, local setup steps (verified working), environment variables guide, testing instructions, what you learned.

Q2. How do you explain a complex technical decision in an interview?

A: Use plain language first: 'Instead of storing passwords directly (dangerous), I use bcrypt to create a one-way hash'. Then explain the WHY: 'because if the database is stolen, attackers can't easily reverse a bcrypt hash'. Technical terms after the concept.

Q3. What metrics should you collect about your project?

A: Lighthouse scores (Performance, Accessibility, SEO), test coverage percentage, API response times (p50, p95), bundle size, time to deploy, number of features shipped. Concrete numbers make your STAR answers credible.

Q4. How do you present a project you built following a tutorial?

A: Be transparent: 'I started from a tutorial but extended it significantly by adding X, Y, Z'. Show what you added. Discuss decisions you made. Explain what you understand about every part. Understanding > originality.

Q5. What is the most common mistake developers make when showcasing projects?

A: Not having a live demo URL. Interviewers don't clone and run your code. If there's no live URL, assume they don't see your project. Deploy everything. Even a simple portfolio on GitHub Pages is better than 'runs locally'.

■ Launch & Showcase Cheat Sheet

Live URL in README badge	First thing recruiters look for
Demo GIF in README	Shows app working in 10 seconds
Tech stack table	Quick scan for recruiters
Setup in 5 commands	Anyone can run it
.env.example committed	Template without real secrets
STAR format prepared	Situation Task Action Result
3 questions per project	Walk through, hardest part, redo
Lighthouse 90+ screenshot	Proof of quality

Pin to GitHub profile	First thing on your GitHub page
LinkedIn post with GIF	Visibility to recruiters
Keep iterating weekly	Shows ongoing commitment
What would you do differently	Self-awareness impresses interviewers